Ders: Web Programcılığı III	**Öğrenci Adı Soyadı:** Hasan Ali YÜCEL
Ödev Konusu: Monolitik/Mikroservis Mimarileri ve Proje Analizi	**Öğrenci No / İmza:** ..... / .....

WEB PROGRAMCILIĞI III — ARAŞTIRMA ÖDEVİ 1

Yazılım Mimarileri (Monolitik & Mikroservis), Yatay/Dikey Ölçeklendirme ve Kapsamlı Proje Analizi

ARAŞTIRMA BÖLÜMÜ - 1: YAZILIM MİMARİLERİ VE ÖLÇEKLENDİRME

1. Monolitik Mimari ve Mikroservis Mimarisi Kavramları

Monolitik Mimari (Monolithic Architecture): Monolitik mimariyi en sade ve anlaşılır haliyle; bir yazılım projesindeki tüm bileşenlerin (kullanıcı arayüzü, iş kuralları mantığı, veritabanı erişim katmanı, yetkilendirme, sepet ve ödeme sistemleri gibi) tek bir kod tabanında (codebase) birleştirildiği, tek bir veritabanı şemasını paylaştığı ve sunucuya tek bir çalıştırılabilir dosya ya da paket (.dll, .war, tek bir container imajı vb.) halinde dağıtıldığı geleneksel mimari model olarak açıklayabilirim. Bu mimaride sınıflar ve katmanlar birbirinden farklı klasörlerde yer alsa bile aynı çalışma zamanı (runtime) ortamında, aynı işletim sistemi prosesinde ve aynı bellek alanında (RAM) birlikte yaşarlar.

Örneğin derste ASP.NET Core MVC ile yazdığımız bir projeyi ele alırsak; Controller sınıfları, DbContext veritabanı bağlantısı, modeller ve View dosyaları tek bir Visual Studio çözümü içindedir. Derle butonuna bastığımızda tek bir çıktı oluşur ve sunucuya yüklenir. Bu modelin en güçlü yönü, projenin ilk geliştirme evresinde ve küçük ekiplerde sağladığı muazzam hız ve kolaylıktır. Bilgisayarımızda tek tıkla hata ayıklama (debugging) yapabiliriz, metotlar arası çağrılar doğrudan RAM üzerinden nanosaniyelerde gerçekleşir ve ağ gecikmesi yaşanmaz. Dağıtım süreci tek bir dosyayı sunucuya kopyalamaktan ibarettir. Ancak proje büyüdükçe, kod hacmi yüz binlerce satırı aştıkça monolitik yapı hantallaşır. Sistemdeki ufacık bir kod hatası (örneğin kontrolsüz bir döngü) tüm sunucuyu kilitleyebilir ve yeni özellik eklemek diğer modülleri bozma riski nedeniyle büyük bir stres haline gelir.

Mikroservis Mimarisi (Microservices Architecture): Mikroservis mimarisi ise monolitik yapının tam tersine, büyük ve karmaşık bir yazılım ekosistemini iş kabiliyetlerine (Business Capabilities) göre sınırlandırılmış, her biri kendi başına bağımsız bir proses olarak çalışan, küçük ve gevşek bağlı (loosely coupled) servis parçacıklarına bölme yaklaşımıdır. Buradaki "mikro" tabiri kodun satır sayısının azlığından ziyade, servisin tek bir işe odaklanmasını (Tek Sorumluluk Prensibi - Single Responsibility) temsil eder.

Bir e-ticaret platformunu mikroservis yaklaşımıyla modellediğimizde; Kullanıcı Servisi, Ürün Servisi, Sepet Servisi, Ödeme Servisi ve Kargo Bildirim Servisi gibi tamamen izole projeler oluştururuz. Bu mimarinin altın kuralı her servisin kendine ait izole bir veritabanına sahip olmasıdır (Database-per-Service). Bir servis asla başka bir servisin veritabanına doğrudan SQL sorgusu atamaz; iletişim yalnızca iyi tanımlanmış hafif siklet ağ protokolleri (HTTP REST API, gRPC veya RabbitMQ/Kafka gibi asenkron mesaj kuyrukları) üzerinden gerçekleştirilir. Bu mimarinin en çarpıcı artısı bağımsız dağıtım (deployment) ve teknoloji özgürlüğüdür (Polyglot). Örneğin ürün arama ve yapay zeka tavsiye motorunu Python ile yazarken, yüksek hız ve eşzamanlılık gerektiren sipariş servisini C# (.NET Core) ile, hızlı bildirim kuyruğunu ise Go veya Node.js ile geliştirebiliriz. Bir serviste güncelleme yapıldığında sadece o servis canlıya alınır, sistemin geri kalanı kesintisiz çalışmaya devam eder. Buna karşılık mikroservisler; ağ gecikmeleri, dağıtık sistemlerde veri

tutarlılığı sağlama zorunluluğu (Saga Pattern ihtiyacı), merkezi log izleme (Elasticsearch/Jaeger) ve servis keşfi gibi ciddi operasyonel karmaşıklıkları da beraberinde getirir.

2. Monolitik Mimaride Yatay ve Dikey Genişleme (Ölçeklendirme) ve Aralarındaki Farklar

Yazdığımız bir monolitik web uygulaması başlangıçta günlük birkaç yüz kullanıcıya rahatça yanıt verebilirken, kampanya zamanlarında veya kullanıcı sayısı hızla arttığında sunucunun donanım kaynakları (CPU, RAM, Disk I/O) tükenir ve sistem ya aşırı yavaşlar ya da "502 Bad Gateway / Connection Timeout" hatalarıyla çöker. Bu darboğazı aşmak için uygulanan iki temel genişleme stratejisi bulunmaktadır:

- **Dikey Genişleme (Vertical Scaling - Scale-Up):** Mevcut uygulamanın koşturulduğu tek bir fiziksel sunucunun veya sanal makinenin donanımsal gücünü artırmaktır. Örneğin 8 GB RAM'e sahip sunucuyu 64 GB veya 128 GB RAM'e çıkarmak, 4 çekirdekli işlemci yerine 32 çekirdekli daha güçlü bir işlemci takmak ve diskleri yüksek hızlı NVMe SSD'lere yükseltmektir. Bunu mevcut tek nakliye kamyonumuzun motorunu büyütmeye ve şasisini güçlendirmeye benzetebiliriz. *Avantajı;* yazılımın kodunda ve mimarisinde hiçbir değişiklik gerektirmemesidir; kod aynı kalır, daha güçlü makinede çalışır. *Dezavantajı;* dikey büyümenin donanımsal ve fiziksel bir sınırı vardır (anakartın ve pazarın sunduğu en yüksek konfigürasyon sınırı). Ayrıca donanım güçlendikçe maliyet üstel (katlanarak) artar. Yükseltme esnasında sunucunun kapatılması gerektiğinden kesinti (downtime) kaçınılmazdır ve sistem hâlâ "Tek Hata Noktası" (Single Point of Failure) durumundadır; tek sunucu bozulursa tüm platform durur.
- **Yatay Genişleme (Horizontal Scaling - Scale-Out):** Tek bir süper bilgisayar inşa etmek yerine, sisteme aynı monolitik uygulamanın kopyalarını çalıştıran birden fazla bağımsız ve uygun maliyetli sunucu (node) dahil etmektir. Bunu filomuza aynı işi yapan yeni ticari araçlar katmaya benzetebiliriz. Yatay genişlemede gelen kullanıcı isteklerini bu sunuculara dengeli dağıtmak için sunucuların önüne mutlaka bir **Yük Dengeleyici (Load Balancer - Nginx, HAProxy, AWS ALB)** yerleştirilir. *Avantajı;* teorik olarak sınırsız ölçekleme imkanı sunması ve yüksek erişilebilirliktir (High Availability). Sunuculardan ikisi çökse bile diğer sunucular trafiği karşılar ve sistem ayakta kalır. Trafik düşünce sunucu sayısı azaltılarak bulut maliyetleri optimize edilir (elastikiyet). *Zorluğu;* yazılımın mutlaka "Durumsuz" (Stateless) tasarlanmış olmasını zorunlu kılmasıdır. Kullanıcı oturum bilgileri (Session) yerel sunucu belleğinde (In-Memory) tutulursa, kullanıcı ikinci istekte başka bir sunucuya yönlendirildiğinde oturumu düşer. Bu nedenle oturumlar Redis gibi harici merkezi bir önbelleğe taşınmalı, yüklenen medya dosyaları ise ortak bir bulut depolama alanına (S3 / Storage) aktarılmalıdır.

Karşılaştırma Kriteri	Dikey Genişleme (Scale-Up)	Yatay Genişleme (Scale-Out)
Temel Yaklaşım	Mevcut tek sunucunun donanımını (CPU, RAM, Disk) artırmak.	Aynı uygulamayı koşturan yeni sunucu kopyaları (node) eklemek.
Maliyet Modeli	Üst segment donanımlarda maliyet astronomik şekilde artar.	Ekonomik, standart donanımlarla doğrusal ve elastik maliyet.
Yazılımsal Değişiklik	Kodda ve mimaride hiçbir değişiklik gerektirmez.	Stateless mimari, Load Balancer ve merkezi Redis oturumu gerekir.
Hizmet Kesintisi	Donanım takviyesi esnasında genellikle sunucu kapatılır (downtime).	Sistem çalışırken yeni sunucu eklenebilir / çıkarılabilir (Zero-Downtime).
Hata Toleransı (SPOF)	Tek hata noktasıdır; sunucu arızalanırsa tüm hizmet kesilir.	Yedekli yapı vardır; çöken sunucunun yükünü diğerleri devralır.

3. Mikroservis ve Monolitik Mimari Arasındaki Farklar

Araştırmalarım ve analizlerim sonucunda iki mimari arasındaki temel ayrımları 5 kritik başlıkta toplayabilirim:

- **1. Kod Tabanı ve Dağıtım (Deployment) Esnekliği:** Monolitte tüm işlevler tek bir repository'de toplanır; küçük bir CSS veya metot değişikliğinde dahi tüm uygulamanın baştan derlenip koca paketin canlıya alınması gerekir. Mikroserviste ise her servisin kendi bağımsız deposu ve CI/CD boru hattı vardır; sadece ilgili servis dakikalar içinde güncellenir.
- **2. Veritabanı ve Veri Bütünlüğü Modeli:** Monolitik mimari tek bir ilişkisel veritabanına bağlıdır; tablolar arası JOIN'ler ve ACID garantili transaction'lar ile veri bütünlüğü çok kolay sağlanır. Mikroserviste ise her servisin kendi izole veri tabanı vardır (Database-per-Service); doğrudan JOIN yapılamaz, veri tutarlılığı anlık değil "Eventual Consistency" (zamanla tutarlı) modeline dayanır ve dağıtık transaction'lar Saga deseniyle yönetilir.
- **3. Hata İzolasyonu ve Çökme Etkisi (Blast Radius):** Monolitik uygulamada fırlatılan ve yakalanmayan kritik bir istisna ya da bellek sızıntısı (memory leak) tüm sunucuyu kilitleyip tüm sistemi çökertebilir. Mikroservis mimarisinde hata alanı izoledir; örneğin "Ürün Yorum Servisi" aşırı yükten çökse dahi "Sepet ve Ödeme Servisleri" bağımsız konteynerlarında sorunsuz çalışmaya devam eder ve satışlar durmaz.
- **4. Ölçeklendirme Hassasiyeti ve Kaynak Tüketimi:** Monolitte sadece ödeme veya arama modülü yoğunlaştığında bile tüm monolit uygulamayı kopyalamak gerekir, bu da atıl modüllerin boş yere kaynak tüketmesine yol açar. Mikroserviste ise yalnızca darboğaz yaşayan servise (örneğin kampanya zamanı Ürün Arama Servisine) yüzlerce pod/kopya atanabilir.
- **5. Teknoloji Çeşitliliği ve Ekip Otonomisi:** Monolitte tüm ekip tek bir dil ve çatıya (.NET, Java vb.) bağımlıdır ve ortak kod üzerinde sürekli git çakışmaları yaşanır. Mikroserviste her servis işe en uygun teknolojiyle (Polyglot) geliştirilebilir ve küçük otonom ekipler (Two-Pizza Teams) birbirini beklemeden paralel çalışabilir.

ARAŞTIRMA BÖLÜMÜ - 2: KAPSAMLI PROJE ANALİZİ

1. Belirlenen Proje ve Genel Tanıtımı

Proje Adı: *"NovaPazar: Çok Satıcılı (Multi-Vendor) B2C E-Ticaret, Dinamik Stok ve Akıllı Sipariş Yönetim Platformu"*

Basit ve statik bir web sitesinin ötesinde; binlerce bağımsız mağazanın kendi ürün varyantlarını yönetebildiği, yüz binlerce tüketicinin anlık filtrelerle arama yapıp güvenli ödeme gerçekleştirdiği, arka planda dinamik stok rezervasyonu, kargo barkodlama ve satıcı hakediş mutabakatı gibi karmaşık kuralları işleten uçtan uca modern bir e-ticaret ekosistemidir.

2. Projenin Amacı

NovaPazar projesinin amacı; yerel işletmeleri, butik üreticileri ve KOBİ'leri modern dijital ticaret altyapısıyla donatarak büyük pazaryeri tekellerine karşı adil komisyon oranlarıyla rekabet edebilir kılmaktır. Sistem; stok çakışmalarını sıfıra indiren akıllı envanter algoritmaları, BDDK uyumlu şeffaf ödeme ve otomatik kargo barkodlama altyapısıyla hem satıcıların operasyonel iş yükünü hafifletmeyi hem de tüketicilere yıldırım hızında, güvenli bir alışveriş deneyimi sunmayı hedefler.

3. Projenin Kapsamı

- **Kapsam Dahilinde Olanlar (In-Scope):** Kullanıcı ve Rol Yönetimi (Alıcı, Satıcı, Admin, Destek - JWT & 2FA), Ürün Kataloğu ve Varyant Yönetimi (Beden/Renk, Elasticsearch anlık arama), Sepet ve Dinamik Promosyon Motoru, Güvenli Ödeme ve Sanal POS Entegrasyonu (Iyzico/PayTR, 3D Secure, taksit), Sipariş ve Dağıtık Stok Yönetimi (Pessimistic/Redis kilitlemeli stok rezervasyonu), Kargo API Entegrasyonu (Aras, MNG, Yurtiçi otomatik takip ve barkod üretimi), Satıcı Yönetim Portalı (Envanter, sipariş, finansal hakediş raporu) ve Yönetici Denetim Paneli.
- **Kapsam Dışında Bırakılanlar (Out-of-Scope):** Fiziksel kurye/araç takip donanımı ve depo raflama altyapısı (kargo firmalarının mevcut web servisleri entegre edilecektir), satıcıların iç muhasebe ERP/üretim sistemleri (yalnızca standart e-Fatura/e-Arşiv XML/PDF aktarım API'si sağlanır) ve sıfırdan bir bankacılık altyapısı kurmak (lisanslı ödeme kuruluşlarının API'leri kullanılacaktır).

4. Projenin Çözümlediği Problem

- **1. Eşzamanlı Stok Çakışması ve Fazladan Satış (Over-Selling):** İndirim dönemlerinde aynı ürünü aynı saniyede birden çok kişinin satın alması sonucu stok eksiye düşmekte ve siparişler zorunlu iptal edilmektedir. NovaPazar, veritabanı kilit mekanizması ve 15 dakikalık sepet rezervasyonu ile bu sorunu tamamen engeller.
- **2. Yüksek Satıcı Komisyonları ve Finansal Nakit Blokajı:** Dev pazaryerlerindeki %25-30 komisyonlar ve 45 günlük hakediş vadeleri KOBİ'lerin nakit akışını tıkamaktadır. Platformumuz, düşük komisyon modeli ve haftalık otomatik mutabakat sunar.
- **3. Alıcı Güvensizliği ve Kargo Takipsizliği:** Müşterilerin sipariş sonrası kargonun nerede olduğunu bilememesi ve sahte yorumlar nedeniyle yaşadığı güvensizlik, şeffaf adım adım bildirim ve onaylı alışveriş yorumu kuralıyla aşılmaktadır.

5. Projenin Hedef Kitlesi

- **Bireysel Alıcılar (Tüketiciler):** İnternet üzerinden uygun fiyatlı, taksitli ve güvenli alışveriş yapmak isteyen 18-65 yaş arası son kullanıcılar (mobil öncelikli arayüz).
- **Mağaza Sahipleri ve Satıcılar (KOBİ'ler):** Fiziksel dükkanını internete taşımak isteyen yerel esnaf, üreticiler ve bağımsız butikler.
- **Platform Operatörleri ve Yöneticiler:** Sistem güvenliğini, mağaza evraklarını, komisyon mutabakatlarını ve yasal uyumu denetleyen şirket içi operasyon ekibi.

6. Proje Zaman Planı (16 Haftalık SDLC Planı)

Faz / Aşama	Zaman Aralığı	Yapılacak Temel Aktiviteler	Teslim Edilecek Çıktılar
Faz 1: Analiz & Gereksinim	Hafta 1 - 3	Paydaş görüşmeleri, pazar araştırması, 24 gereksinimin MoSCoW sınıflandırması ve Use Case modelleri.	Gereksinim Belirtim Dokümanı (SRS), Use Case Şemaları.
Faz 2: Tasarım & Mimari	Hafta 4 - 6	Veritabanı ER diyagramı, mikroservis/API mimarisi, Figma kullanıcı deneyimi (UI/UX) ekran tasarımları.	Veritabanı Şeması, Figma Ekranları, Mimari Tasarım Raporu.
Faz 3: Geliştirme (Sprintler)	Hafta 7 - 12 (6 Hafta)	Sprint 1: Auth & Roller Sprint 2: Katalog & Arama Sprint 3: Sepet & Ödeme Sprint 4: Sipariş, Kargo & Panel	Çalışan Kaynak Kodlar, REST API Uç Noktaları, Yönetim Paneli.
Faz 4: Test & Doğrulama	Hafta 13 - 14	Birim (Unit) testleri, Sanal POS simülasyonları, JMeter yük/stres testleri ve Kullanıcı Kabul Testleri (UAT).	Test Sonuç Raporu, Hata Giderme Günlüğü, Güvenlik Raporu.
Faz 5: Dağıtım & Kapanış	Hafta 15 - 16	Bulut sunucu kurulumu (Docker/K8s), CI/CD devreye alma, satıcı kullanım kılavuzları ve Go-Live.	Canlıda Çalışan Sistem, Satıcı Kılavuzu, Proje Kapanış Sunumu.

7. Proje Gereksinimleri (24 Adet) ve MoSCoW Sınıflandırması

Kod	Gereksinim Tanımı ve Detayı	MoSCoW Sınıfı	Öncelik Gerekçesi
G01	Kullanıcı Kayıt, Giriş ve Rol Tabanlı Kimlik Doğrulama (JWT & bcrypt).	**Must Have (M)**	Sistem güvenliği ve alıcı-satıcı rollerinin ayrılması için temel altyapı.
G02	Ürün Ekleme, Varyant Tanımlama (Beden, Renk) ve Çoklu Görsel Yükleme.	**Must Have (M)**	E-ticaretin varlık sebebi; ürün olmadan satış ve katalog var olamaz.
G03	Kategori Hiyerarşisi ve Dinamik Arama / Filtreleme Motoru.	**Must Have (M)**	Kullanıcının aradığı ürünü bulabilmesi platformun en temel fonksiyonudur.
G04	Sepet Yönetimi ve Eşzamanlı Dinamik Stok Rezervasyon Mekanizması.	**Must Have (M)**	Satın alma kararı ve mükerrer satışı (over-selling) önlemek için zorunlu.
G05	BDDK Lisanslı Sanal POS Entegrasyonu ve 3D Secure ile Ödeme Tahsilatı.	**Must Have (M)**	Ticari gelirin elde edilmesi ve yasal mevzuata uyum için şarttır.
G06	Sipariş Oluşturma, Sipariş Özeti ve Müşteri Durum Takip Ekranı.	**Must Have (M)**	Ödeme sonrası yasal mesafeli satış sözleşmesi ve tüketici hakkı için zorunlu.
G07	Müşteri Sipariş İptali ve Otomatik Para İadesi / Stok Yenileme Süreci.	**Must Have (M)**	Tüketici Hakları Kanunu gereği yasal zorunluluk ve operasyonel ihtiyaç.
G08	Satıcı Mağaza Başvuru, Vergi Levhası Doğrulama ve Yönetici Onayı.	**Must Have (M)**	Pazaryerinde dolandırıcılığı önlemek ve yasal satıcılarla çalışmak için şart.
G09	Kargo Firmaları API Entegrasyonu (Otomatik Barkod ve Takip Kodu).	**Should Have (S)**	Sipariş teslimatının manuel takibini önleyip operasyonu hızlandırır.
G10	Dinamik İndirim Kuponu, Promosyon ve Kampanya Motoru.	**Should Have (S)**	Pazarlama ve müşteri sadakatini artırmak için çok kritik fonksiyondur.
G11	Yalnızca Ürünü Satın Alan Onaylı Müşterilerin Yorum ve Puan Verebilmesi.	**Should Have (S)**	Sahte yorumları engelleyerek platform güvenilirliğini üst seviyeye çıkarır.
G12	Sipariş Durum Değişikliklerinde Otomatik SMS ve E-posta Bildirimi.	**Should Have (S)**	Müşteri bilgilendirmesi ve çağrı merkezi yükünü azaltmak için gereklidir.
G13	Satıcı Mağaza Finansal Paneli (Satışlar, Komisyon Kesintisi ve Bakiye).	**Should Have (S)**	Satıcıların finansal şeffaflığı görmesi ve mutabakat için önemlidir.
G14	Favori Ürünler Listesi (Wishlist) ve Fiyatı Düşünce Haber Ver Alarmı.	**Should Have (S)**	Kullanıcı etkileşimini ve satışa dönüşüm oranlarını artırır.

Kod	Gereksinim Tanımı ve Detayı	MoSCoW Sınıfı	Öncelik Gerekçesi
G15	Sosyal Medya Hesapları (Google, Apple) ile Hızlı Tek Tıkla Giriş (OAuth2).	**Could Have (C)**	Kayıt olma sürtünmesini azaltır, ilk sürümde olmasa da sistem yürür.
G16	Yapay Zeka Tabanlı 'Bunu Alanlar Şunları da Aldı' Tavsiye Motoru.	**Could Have (C)**	Sepet ortalama tutarını artırır ancak ilk açılışta zorunlu değildir.
G17	Canlı Sohbet (Live Chat) ve Müşteri Destek Talep (Ticket) Sistemi.	**Could Have (C)**	Destek e-posta ile de yürütülebilir, zaman kalırsa eklenecektir.
G18	Çoklu Para Birimi (USD, EUR) ve Çok Dilli Arayüz (İngilizce/Türkçe).	**Could Have (C)**	Yurt dışı açılımı için iyidir, yerel başlangıçta şart değildir.
G19	PWA (Progressive Web App) Desteği ile Mobil Tarayıcıdan Yüklenme.	**Could Have (C)**	Mobil kullanıcı deneyimini iyileştirir fakat responsive tasarım yeterlidir.
G20	Sesli Komut ile Ürün Arama ve Sesli Alışveriş Asistanı Entegrasyonu.	**Won't Have (W)**	Mevcut faz için yüksek maliyetli ve karmaşık; sonraki sürümlere bırakıldı.
G21	Kripto Para (Bitcoin, USDT) ile Anonim Ödeme Seçeneği.	**Won't Have (W)**	Mevcut yasal regülasyonlar ve vergi mevzuatı nedeniyle kapsam dışıdır.
G22	Satıcı Kuryeleri İçin Canlı GPS Harita Üzerinde Canlı Kurye Takibi.	**Won't Have (W)**	Yemek dağıtım platformu değiliz, standart kargo API'si yeterlidir.
G23	Fiziksel Mağazalar İçin Donanımsal Masaüstü Kasa Satış (POS) Sistemi.	**Won't Have (W)**	Proje sadece dijital web ortamına odaklanmıştır, masaüstü POS yapılmayacaktır.
G24	Artırılmış Gerçeklik (AR) ile Odada Ürün Önizleme / Sanal Deneme.	**Won't Have (W)**	Teslim takvimini aşacağı için kesinlikle sonraki yıllara ertelenmiştir.

8. Gereksinimlerin Türlerine Göre Yeniden Sınıflandırılması

A) İş Gereksinimleri (Business Requirements)

Platformu kuran işletmenin ticari hedeflerini, kârlılık stratejisini ve yasal uyumunu tanımlar:

- **G05 (Ödeme ve Komisyon Kesintisi):** Platformun ticari sürdürülebilirliğini sağlamak ve satılan ürünlerden %8-12 oranında komisyon kesintisini otomatik tahsil etmek.
- **G08 (Satıcı Doğrulama ve Yasal Uyum):** Satıcıların vergi levhası ve IBAN bilgilerini doğrulayarak Ticaret Bakanlığı ETBİS kurallarına tam uyum sağlamak.
- **G13 (Satıcı Hakediş Mutabakatı):** Haftalık satıcı hakedişlerini şeffaf biçimde hesaplayarak platform ile mağazalar arasındaki nakit akışını hatasız yürütmek.
- **G17 (Müşteri Memnuniyeti ve İhtilaf Çözümü):** Müşteri şikayetlerini yasal süre içinde çözüme kavuşturarak platform marka güvenilirliğini korumak.

- **G21 (Yasal Mevzuat ve Risk Sınırı):** Finansal regülasyonlar izin vermediği ve vergi takibi yapılamadığı için kripto para ile ödemeyi ilk aşamada kesin olarak reddetmek.

B) Kullanıcı Odaklı Gereksinimler (User Requirements)

Alıcı ve satıcıların sistemi kullanırken ne beklediğini ve nasıl bir deneyim yaşamak istediğini tanımlar:

- **G01 & G15 (Hesap Yönetimi ve Hızlı Giriş):** Alıcılar tek bir formla veya Google/Apple ile hızlıca üye olabilmeli, şifrelerini güvenle sıfırlayabilmelidir.
- **G03 (Kolay Ürün Arama ve Filtreleme):** Alıcılar ilgilendikleri ürünleri binlerce seçenek arasından kategori, fiyat aralığı, marka ve kullanıcı puanına göre anında süzebilmelidir.
- **G09 (Şeffaf Kargo Takip İhtiyacı):** Müşteriler sipariş ettikleri ürünün hangi kargo şubesinde ve hangi dağıtım aşamasında olduğunu canlı olarak görebilmelidir.
- **G11 (Doğrulanmış Müşteri Yorumları):** Alıcılar yalnızca ürünü gerçekten satın almış kişilerin yaptığı doğrulanmış yorumlara ve fotoğraflara güvenebilmelidir.
- **G14 (Favori Listesi ve Fiyat Alarmı):** Kullanıcılar beğendikleri ürünleri favoriye alıp indirim olduğunda otomatik haber almak istemektedir.

C) Fonksiyonel Gereksinimler (Functional Requirements)

Sistemin gerçekleştirmesi gereken somut teknik süreçleri, algoritmaları ve girdi-çıktı mantığını tanımlar:

- **G02 (Ürün ve Varyant Girişi):** Satıcı panelinde girilen beden, renk, stok adedi ve görsellerin veritabanına ve bulut depolama sunucusuna işlenmesi.
- **G04 (Sepet ve Dinamik Stok Rezervasyonu):** Kullanıcı sepete ürün eklediğinde veritabanında "ReservedStock" alanının artırılması ve 15 dakikalık geri sayım başlatılması.
- **G05 & G06 (Sanal POS Ödeme ve Sipariş Kaydı):** Sanal POS API'sine SHA-512 şifreli hash parametreleriyle istek atılması, bankadan dönen 3D callback sonucunun doğrulanması ve kalıcı sipariş kaydı üretilmesi.
- **G07 (İptal, Para İadesi ve Stok Restorasyonu):** Kargoya verilmemiş siparişlerde iptal butonuna basıldığında banka iade API'sinin tetiklenmesi, sipariş durumunun "İptal Edildi"ye çekilmesi ve stok sayacının otomatik 1 artırılması.
- **G10 (Kupon ve İndirim Motoru):** Sepet toplamında tanımlı kampanya kurallarının (ör. "500 TL üzeri 50 TL indirim") ve kupon kodunun matematiksel olarak uygulanması.
- **G12 (Otomatik Bildirim Motoru):** Sipariş durumu değiştiğinde asenkron kuyruk üzerinden müşteriye SMS ve şablon e-posta tetiklenmesi.

D) Fonksiyonel Olmayan Gereksinimler (Non-Functional Requirements)

Sistemin genel kalite niteliklerini, performans, güvenlik ve ölçeklenebilirlik standartlarını tanımlar:

- **Performans ve Hız Kriteri:** Sistem mimarisi anlık 10.000 eşzamanlı aktif kullanıcıyı desteklemeli; ürün listeleme ve arama sorguları ortalama 200 milisaniyenin altında yanıt vermelidir.
- **Güvenlik ve Gizlilik (Security):** Kredi kartı verileri kesinlikle sunucularda saklanmayacaktır (PCI-DSS Uyumlu). Şifreler en az 12 tuzlama faktörlü bcrypt algoritması ile hash'lenecektir. Tüm ağ trafiği TLS 1.3 / HTTPS üzerinden şifrelenecektir.
- **Güvenilirlik ve Yedeklilik (Reliability):** Sistem yılda en az %99.9 erişilebilirlik (High Availability) sağlamalıdır. Her gece saat 03:00'te otomatik artımlı veritabanı yedeği şifreli olarak harici buluta alınmalıdır.

- **Kullanılabilirlik (Usability):** Arayüz responsive yapıda olmalı; masaüstü, tablet ve mobil ekranlarda akıcı bir şekilde çalışmalıdır.

9. Seçilen 3 Fonksiyonel Gereksinim İçin Kullanım Senaryoları (Use Cases)

KULLANIM SENARYOSU 1 (UC-01): Sepete Ürün Ekleme ve Dinamik Stok Kontrolü

UC-01: Sepete Ürün Ekleme ve Anlık Stok Rezervasyonu	
İlişkili Gereksinim:	G04 (Must Have - Fonksiyonel Gereksinim)
Aktörler:	Birincil Aktör: Alıcı (Kayıtlı veya Misafir) | İkincil Aktör: Envanter/Stok Yönetim Servisi
Ön Koşullar:	1. Kullanıcı ürün detay sayfasını tarayıcısında açmış olmalıdır. 2. İlgili ürün varyantına (ör. Beden: M, Renk: Siyah) ait sistemde aktif stok bulunmalıdır.
Son Koşullar:	1. Ürün kullanıcının sepetine eklenir ve sepet tutarı yeniden hesaplanır. 2. Veritabanındaki 'AvailableStock' düşülüp 'ReservedStock' 15 dakika süreliğine 1 artırılır.
Tetikleyici:	Kullanıcının ürün detay sayfasındaki "Sepete Ekle" butonuna tıklaması.
Ana Başarı Akışı (Happy Path):	1. Kullanıcı ürün detay sayfasında istediği varyantı (ör. Ayakkabı Numarası: 42) ve adedi seçer. 2. Kullanıcı "Sepete Ekle" butonuna basar. 3. Sistem, seçilen ürün varyantının mevcut stok miktarını veritabanından asenkron olarak sorgular. 4. Stok miktarının istenen adede eşit veya büyük olduğu doğrulanır. 5. Sistem, seçilen ürünü kullanıcının sepet listesine ekler (sepette zaten varsa adedini günceller). 6. Sistem, ürünün rezerve edilme sayacını başlatır (15 dakikalık Redis TTL anahtarı oluşturur). 7. Sistem sepet ara toplamını, KDV tutarını ve kargo bedelini yeniden hesaplar. 8. Kullanıcı arayüzünde "Ürün Başarıyla Sepetinize Eklendi" bildirim kutusu gösterilir ve sağ üstteki sepet sayacı güncellenir.
Alternatif ve İstisnai Akışlar:	**A1 - Stok Yetersizliği / Tükendi Durumu (Adım 4'te):** a) Kullanıcı butona bastığında stok 0 ise işlem durdurulur. b) Ekranda "Seçtiğiniz ürünün stoğu az önce tükenmiştir." uyarısı gösterilir ve sepete ekleme engellenir. **A2 - Maksimum Adet Aşımı (Adım 4'te):** a) Kullanıcı aynı üründen sepete 10 adetten fazla eklemek isterse sistem izin vermez. b) "Bireysel alımlarda bir üründen en fazla 10 adet eklenebilir." uyarısı verilir. **A3 - Rezervasyon Zaman Aşımı (Adım 6 sonrasında):** a) Kullanıcı 15 dakika boyunca satın alma adımına geçmezse Redis rezervasyon anahtarını düşürür. b) Ürün sepetten silinmez ancak "Stok rezervasyon süreniz doldu, stok garantisi verilmemektedir" bilgisi belirir.

KULLANIM SENARYOSU 2 (UC-02): Sanal POS ile 3D Secure Güvenli Ödeme ve Sipariş Onayı

UC-02: Sanal POS ile 3D Secure Güvenli Ödeme ve Sipariş Oluşturma	
İlişkili Gereksinim:	G05 & G06 (Must Have - Fonksiyonel Gereksinim)
Aktörler:	Birincil Aktör: Alıcı | İkincil Aktörler: Sanal POS Ödeme Ağ Geçidi (Iyzico/Banka), Finans Servisi
Ön Koşullar:	1. Kullanıcı sisteme giriş yapmış ve sepetinde en az bir ürün bulunmalıdır. 2. Kullanıcı teslimat ve fatura adresini seçmiş, mesafeli satış sözleşmesini onaylamış olmalıdır.
Son Koşullar:	1. Ödeme tutarı kullanıcının kredi kartından başarıyla tahsil edilir. 2. Kalıcı sipariş kaydı üretilir, rezerve stoklar kesin olarak stoktan düşülür ve onay e-postası yollanır.
Tetikleyici:	Kullanıcının ödeme ekranında kart bilgilerini girip "Siparişi Onayla ve Öde" butonuna tıklaması.
Ana Başarı Akışı (Happy Path):	1. Kullanıcı kredi kartı üzerindeki 16 haneli kart numarasını, son kullanma tarihini ve CVV kodunu girer. 2. Sistem, kartın BIN numarasından bankayı ve taksit seçeneklerini sorgular; kullanıcı varsa taksit seçer. 3. Kullanıcı "Ödemeyi Tamamla" butonuna basar. 4. Sistem arka planda PCI-DSS standartlarına uygun şifrelenmiş hash paketi oluşturarak Sanal POS API'sine "3D Secure Başlatma" isteği gönderir. 5. Kullanıcı bankanın güvenli 3D Secure SMS doğrulama sayfasına yönlendirilir. 6. Kullanıcı cep telefonuna gelen SMS tek kullanımlık şifresini (OTP) banka ekranına girip onaylar. 7. Banka işlemi onaylar ve platformun güvenli Callback API uç noktasına şifreli onay belirteci iletir. 8. Sistem dönen imzayı doğrular, ödeme tutarını hesaba işler ve "Sipariş Durumu = Onaylandı" olarak kaydeder. 9. Rezerve stoklar kalıcı olarak düşürülür, sepet boşaltılır ve kullanıcıya referans sipariş takip numarası gösterilir.
Alternatif ve İstisnai Akışlar:	**A1 - Yetersiz Bakiye veya Hatalı CVV (Adım 7'de):** a) Banka "051 - Yetersiz Bakiye" veya "Hatalı Güvenlik Kodu" yanıtı döner. b) Sistem işlemi durdurur, sipariş oluşturulmaz ve kullanıcıya kırmızı hata mesajı gösterilir. **A2 - SMS Şifresinin Yanlış Girilmesi veya Zaman Aşımı (Adım 6'da):** a) Kullanıcı 3 dakika içinde SMS kodunu girmezse banka oturumu sonlandırır. b) Sistem kullanıcıyı güvenle ödeme sayfasına geri döndürür, stok rezervasyonu korunur. **A3 - Ağ Kesintisi veya Callback Hatası (Adım 7-8'de):** a) Banka parayı çektiği halde platforma dönüşte ağ koparsa, asenkron mutabakat servisi bankayı sorgular ve mükerrer çekimi engeller.

KULLANIM SENARYOSU 3 (UC-03): Müşteri Sipariş İptali ve Otomatik Para İadesi

UC-03: Müşteri Sipariş İptali ve Otomatik Para İadesi / Stok Restorasyonu	
İlişkili Gereksinim:	G07 (Must Have - Fonksiyonel Gereksinim)
Aktörler:	Birincil Aktör: Alıcı | İkincil Aktörler: Sanal POS İade Servisi, Stok Yönetimi, Satıcı
Ön Koşullar:	1. Kullanıcının oluşturulmuş ve ödemesi tamamlanmış aktif bir siparişi bulunmalıdır. 2. Sipariş durumu "Kargoya Verildi" aşamasına geçmemiş olmalıdır ("Hazırlanıyor" veya "Onaylandı" durumunda olmalıdır).
Son Koşullar:	1. Sipariş statüsü "İptal Edildi"ye güncellenir. 2. Sanal POS API üzerinden karta iade işlemi başlatılır ve ürün stoğu otomatik 1 artırılarak sisteme geri kazandırılır.
Tetikleyici:	Kullanıcının "Siparişlerim" sayfasındaki ilgili sipariş için "Siparişi İptal Et" butonuna basması.
Ana Başarı Akışı (Happy Path):	1. Kullanıcı "Hesabım > Siparişlerim" sayfasına gider ve iptal etmek istediği siparişi seçer. 2. Kullanıcı "Siparişi İptal Et" butonuna tıklar. 3. Sistem siparişin güncel durumunu kontrol eder; henüz satıcı tarafından kargoya verilmediğini teyit eder. 4. Kullanıcıya açılır pencerede iptal gerekçesi sorulur (ör. "Yanlış beden seçtim", "Vazgeçtim"); kullanıcı gerekçeyi seçip onaylar. 5. Sistem veritabanında sipariş durumunu derhal "İptal İşlemi Başlatıldı" durumuna alır. 6. Finans servisi, orijinal ödeme işlem referans kodunu kullanarak Sanal POS API'sine otomatik "Refund / İptal" çağrısı gönderir. 7. Sanal POS sağlayıcısı iadenin başarıyla onaylandığını bildirir. 8. Sistem, sipariş içindeki ürünlerin stok miktarlarını satıcının envanter tablosunda otomatik olarak geri artırır (+1). 9. Siparişte kullanılan indirim kuponu varsa kupon müşterinin hesabına yeniden aktif edilerek tanımlanır. 10. Kullanıcıya SMS ve E-posta ile iptal dekontu ve iade bilgilendirmesi gönderilir.
Alternatif ve İstisnai Akışlar:	**A1 - Ürün Zaten Kargoya Verilmişse (Adım 3'te):** a) Kullanıcı butona bastığında satıcı kargo barkodunu okutmuşsa sistem işlemi reddeder. b) "Siparişiniz kargoya teslim edildiği için iptal edilemez. Ürün ulaştığında 14 gün içinde Kolay İade talebi açabilirsiniz." uyarısı verilir. **A2 - Banka İade API'sinde Geçici Ağ Kesintisi (Adım 6-7'de):** a) Banka API'sine ulaşılamazsa işlem başarısız sayılmaz; sipariş "İade Kuyruğunda (Pending Refund)" statüsüne alınır. b) Arka plan servisi (Hangfire / Quartz) her 15 dakikada bir otomatik yeniden dener ve başarıyla sonuçlandırır.

KAYNAKLAR

Bu araştırma ve analiz çalışmasının hazırlanmasında yararlanılan akademik kaynaklar, uluslararası standartlar ve resmi dokümantasyonlar:

1. **Newman, S. (2021).** *Building Microservices: Designing Fine-Grained Systems (2nd Edition).* O'Reilly Media.
2. **Fowler, M. (2014).** *Microservices: A definition of this new architectural term.* martinfowler.com/articles/microservices.html (Erişim Tarihi: 26.10.2026)
3. **Sommerville, I. (2016).** *Software Engineering (10th Edition).* Pearson Education Limited.

4. **Clegg, D., & Barker, R. (1994).** *Case Method Fast-Track: A RAD Approach (MoSCoW Metodolojisinin Temelleri).* Addison-Wesley.
5. **Microsoft Learn. (2025).** *Cloud Design Patterns: Prescriptive architecture guidance for building on Azure.* learn.microsoft.com/en-us/azure/architecture/patterns/ (Erişim Tarihi: 25.10.2026)
6. **Amazon Web Services (AWS). (2025).** *Microservices on AWS: Architectural Principles and Patterns Whitepaper.* aws.amazon.com/microservices/ (Erişim Tarihi: 27.10.2026)
7. **Cockburn, A. (2000).** *Writing Effective Use Cases (Agile Software Development Series).* Addison-Wesley Professional.
8. **Martin, R. C. (2017).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.

Öğrenci Beyanı:
Bu ödevde yer alan tüm açıklamalar, mimari analizler ve proje senaryoları kendi cümlelerim ve araştırmalarım doğrultusunda hazırlanmıştır.

Teslim Eden Öğrenci:
Hasan Ali YÜCEL
İmza: .....
Tarih: 29.10.2026